CAPSTONE PROJECT REPORT

(Project Term January-May 2026)

Real Time Captioning System

Submitted by

Rishav Vashisht	**Registration Number: 12215349**
Praful Bhardwaj	**Registration Number: 12218952**
Himanshu Shah	**Registration Number: 12210486**
Nurul Islam	**Registration Number: 12206501**

Project Group Number : 2RGCO683

Course Code : CSE439

Under the Guidance of

(Ms. Surabhi Sonkar , Assistant Professsor)

School of Computer Science and Engineering

LOVELY PROFESSIONAL UNIVERSITY

Transforming Education Transforming India

PAC Form

TOPIC APPROVAL PERFORMA

School of Computer Science and Engineering

Program : P132::B.Tech. (Computer Science and Engineering)

COURSE CODE : CSE439 **REGULAR/BACKLOG :** Regular **GROUP NUMBER :** 2RGC0683

Supervisor Name : Surabhi Sonkar **UID :** 34018 **Designation :** Assistant Professor

Qualification : _____ **Research Experience :** _____

SR.NO.	NAME OF STUDENT	Prov. Regd. No.	BATCH	SECTION
1	Praful Bhardwaj	12218952	2022	222WX
2	Rishav Vashisht	12215349	2022	422WP
3	Himanshu Shah	12210486	2022	422PK
4	Nurul Islam	12206501	2022	222ZD

SPECIALIZATION AREA : Artificial Intelligence and Machine Learning **Supervisor Signature:** _____

PROPOSED TOPIC : Real-Time Multilingual Simplified Captioning System

Qualitative Assessment of Proposed Topic by PAC		
Sr.No.	**Parameter**	**Rating (out of 10)**
1	Project Novelty: Potential of the project to create new knowledge	8.00
2	Project Feasibility: Project can be timely carried out in-house with low-cost and available resources in the University by the students.	8.00
3	Project Academic Inputs: Project topic is relevant and makes extensive use of academic inputs in UG program and serves as a culminating effort for core study area of the degree program.	8.00
4	Project Supervision: Project supervisor's is technically competent to guide students, resolve any issues, and impart necessary skills.	8.00
5	Social Applicability: Project work intends to solve a practical problem.	8.00
6	Future Scope: Project has potential to become basis of future research work, publication or patent.	8.00

PAC Committee Members		
PAC Member (HOD/Chairperson) Name: Dr. Mohit Arora	UID: 15980	Recommended (Y/N): Yes
PAC Member (Allied) Name: Dr. Ankita Wadhawan	UID: 23891	Recommended (Y/N): Yes
PAC Member 3 Name: Upinder Kaur	UID: 27954	Recommended (Y/N): Yes

Final Topic Approved by PAC: **Real-Time Multilingual Simplified Captioning System**

Overall Remarks: Approved

PAC CHAIRPERSON Name: 11265::Dr. Dalwinder Singh **Approval Date:** 16 Dec 2025

4/29/2026 3:11:44 PM

DECLARATION

We hereby declare that the project work entitled "Real Time Captioning System" is an authentic record of our own work carried out as requirements of Capstone Project for the award of B.Tech degree in Computer Science and Engineering from Lovely Professional University, Phagwara, under the guidance of Ms. Surabhi Sonkar during January to May 2026. All the information furnished in this capstone project report is based on our own intensive work and is genuine.

Project Group Number: 2RGCO683

Name of Student 1: Praful Bhardwaj
Registration Number: 12218952

Name of Student 2: Rishav Vashisht
Registration Number: 12215349

Name of Student 3: Nurul Islam
Registration Number: 12206501

Name of Student 4: Himanshu Shah
Registration Number: 12210486

Date: Signature of student 1

Date: Signature of student 2

Date: Signature of student 3

Date: Signature of student 4

CERTIFICATE

This is to certify that the declaration statement made by this group of students is correct to the best of my knowledge and belief. They have completed this Capstone Project under my guidance and supervision. The present work is the result of their original investigation, effort and study. No part of the work has ever been submitted for any other degree at any University. The Capstone Project is fit for the submission and partial fulfillment of the conditions for the award of B.Tech degree in Computer Science and Engineering from Lovely Professional University, Phagwara.

Signature and Name of the Mentor : Ms. Surabhi Sonkar

Designation : Assistant Professor

School of Computer Science and Engineering,
Lovely Professional University,
Phagwara, Punjab.

Date : 27-04-2026

ACKNOWLEDGEMENT

First and foremost, We express our deep gratitude to our supervisors, Ms. Surabhi Sonkar, for their unwavering support, insightful feedback, and expert guidance throughout the development of this capstone project. Their domain expertise in speech processing and assistive technology helped shape both the research direction and the engineering decisions that underpin this system.

We are grateful to the Department of Computer Science and Engineering at Lovely Professional University for providing access to computational resources, technical infrastructure, and a stimulating academic environment that made this work possible. The support of the university library and online databases in providing access to research literature was invaluable.

We acknowledge the open-source communities behind the Sarvam AI platform, Supabase, Vite, React, and the Radix UI component library. These tools formed the foundational building blocks of the implementation and exemplify the transformative power of collaborative, open software development.

We would also like to thank the deaf and hard-of-hearing community whose real-world challenges motivated this research, and the researchers whose published work in automatic speech recognition, speaker diarization, and accessibility technology provided the scientific foundation on which this project is constructed.

Finally, we are deeply thankful to my family and colleagues for their patience, encouragement, and moral support throughout this academic journey.

Praful Bhardwaj
Rishav Vashisht
Nurul Islam
Himanshu Shah
Lovely Professional University, April 2026

ABSTRACT

The rapid adoption of video conferencing and Voice-over-Internet-Protocol (VoIP) communication platforms in today's workplaces has given rise to a significant access barrier for the 430 million people globally with disabling hearing loss. For deaf and hard-of-hearing (DHH) professionals, the ability to participate in dynamic spoken interactions - be it for remote work meetings, online training, or even client meetings on the telephone - is made impossible without the availability of fast and accurate captioning.

This report outlines the design, implementation and evaluation of a real-time multilingual captioning system designed to overcome this barrier. The system comprises a six-stage real-time processing pipeline: audio signal capture and voice activity detection, front-end noise reduction with RNNoise, acoustic feature extraction, automatic speech recognition (ASR) using the Sarvam AI speech-to-text engine with CTC prefix beam search, text simplification and translation using the Sarvam AI platform, and accessible rendering of captions in a web interface.

The accompanying web interface - the CaptionAI Hub - is a React and TypeScript-based single-page application built with the Vite build tool, Supabase backend-as-a-service for real-time data storage and edge computing, and Sarvam AI APIs for multilingual speech-to-text, neural machine translation, and text-to-speech synthesis across Hindi, Bengali, Tamil, Telugu, Kannada, Malayalam, Marathi, Gujarati, Punjabi, Odia, and English.

We project system performance based on published benchmarks for the component technologies to be a Word Error Rate (WER) of 2.4% for clean audio and 6.1% for 5 dB signal-to-noise ratio, with a mean caption delay of 210 milliseconds - comfortably below the 300 millisecond threshold for perceptually synchronised captioning identified in the research on DHH captioning. ECAPA-TDNN embeddings facilitate speaker diarisation for up to six speakers. The system is intended for hardware-agnostic implementation as a web extension, with no special hardware or assistance from hearing callers required.

The project's key innovations include: (1) a five-stage, production-ready ASR pipeline with noise suppression, real-time inference, language model rescoring and speaker diarisation; (2) a multilingual front-end for code-switching between Indian language pairs; (3) an AI-powered conversational chatbot for DHH users in their native languages; and (4) an extensive web platform showcasing the system concept with interactive demos.

Keywords: Automatic Speech Recognition, Real-Time Captioning, Deaf Accessibility, Deep Learning, Speaker Diarisation, Multilingual NLP, Sarvam AI, Indian Languages, Assistive Technology, Code-Switching.

TABLE OF CONTENTS

Inner First Page 1

PAC Form 2

Declaration 3

Certificate 4

Acknowledgement 5

Table of Contents 7

CHAPTER 1: INTRODUCTION 11
1.1 Background and Motivation 11
1.2 Project Scope 12
1.3 Objectives 13
1.4 Report Organisation 14
CHAPTER 2: LITERATURE REVIEW 15
2.1 Evolution of Automatic Speech Recognition 15
2.1.1 Statistical Acoustic Models 15
2.1.2 Deep Neural Network Acoustic Models 15
2.1.3 Attention Mechanisms and Conformer Architecture 15
2.2 Large-Scale Pre-Trained Speech Models 16
2.3 Streaming and Real-Time ASR 16
2.4 Captioning Research for DHH Users 17
2.5 Speaker Diarisation 17
2.6 Noise Robustness and Language Model Integration 18
2.7 Multilingual and Indian Language Speech Processing 18
2.8 Comparative Analysis of Existing Solutions 19
CHAPTER 3: PROBLEM STATEMENT 20
3.1 Problem Definition 20
3.2 Technical Challenges 20
3.2.1 Transcription Accuracy in Professional Domains 20
3.2.2 Caption Delivery Speed 20
3.2.3 Speaker Attribution in Group Calls 21
3.2.4 Robustness in Realistic Acoustic Conditions 21
3.3 Indian Language Context 21
3.4 Privacy and Platform Independence Requirements 21
CHAPTER 4: SYSTEM METHODOLOGY AND DESIGN 22
4.1 System Architecture Overview 22
4.2 ASR Pipeline — Stage-by-Stage Description 22
4.2.1 Stage 1: Audio Capture and Voice Activity Detection 22
4.2.2 Stage 2: Noise Reduction and Signal Normalisation 23
4.2.3 Stage 3: Acoustic Feature Extraction 23
4.2.4 Stage 4: ASR Engine with Adaptive Chunking and LM Rescoring 24
4.2.5 Stage 5: Speaker Diarisation and Caption Display 25
4.3 Pseudo-Code: Core Pipeline Algorithm 25
4.4 Web Application — CaptionAI Hub 26
4.4.1 Technology Stack 26
4.4.2 Application Component Architecture 26

4.4.3 AIAssistant Component 27
4.4.4 TranslatorAssistant Component 28
4.4.5 LiveCaptionDemo Component 28
4.4.6 Supabase Edge Function — Chat Endpoint 29
4.4.7 Language Detection Algorithm 29
4.5 System Data Flow 30
CHAPTER 4A: SYSTEM DESIGN
4A.1 Data Flow Diagram (DFD)
4A.2 Entity-Relationship (E-R) Diagram
4A.3 System Flowchart
CHAPTER 5: RESULTS AND EVALUATION 31
5.1 Evaluation Methodology 31
5.2 Word Error Rate Results 31
5.3 Projected Latency vs. Noise Level 32
5.4 Vocabulary Robustness 33
5.5 Ablation Study 33
5.6 Web Application Performance 34
CHAPTER 6: DISCUSSION 35
6.1 Performance Against Design Requirements 35
6.2 Design Decision Rationale 35
6.2.1 Choice of Sarvam AI for ASR 35
6.2.2 Choice of Sarvam AI for Indian Languages 35
6.2.3 Choice of React and Supabase for the Web Application 35
6.3 Comparison with Commercial Solutions 36
6.4 Limitations 36
6.5 Ethical Considerations 37
CHAPTER 7: CONCLUSION AND FUTURE SCOPE 39
7.1 Conclusion 39
7.2 Future Scope 39
7.2.1 Full Prototype Implementation and User Validation 40
7.2.2 Multilingual and Code-Switching Extension 40
7.2.3 Knowledge Distillation for Mobile Devices 40
7.2.4 Multi-Modal Enhancement via Lip Reading 40
7.2.5 Bidirectional Communication with Sign Language 40
CHAPTER 8: REFERENCES 42
APPENDIX A: SYSTEM REQUIREMENTS SPECIFICATION 44
A.1 Hardware Requirements 44
A.2 Software Requirements 44
A.3 Supported Languages 44
APPENDIX B: GLOSSARY OF TERMS 45

LIST OF FIGURES

Figure 1.1 — Growth in DHH population globally (WHO, 2021) 10
Figure 2.1 — Evolution of ASR from HMM-GMM to Transformer models 11
Figure 3.1 — Comparison of WER across commercial captioning platforms 12
Figure 4.1 — Five-stage real-time captioning pipeline architecture 13
Figure 4.2 — Adaptive chunk-size mechanism workflow 14
Figure 4.3 — Speaker diarisation using ECAPA-TDNN and BHM clustering 15
Figure 4.4 — CaptionAI Hub application architecture 16
Figure 4.5 — Sarvam AI integration data-flow diagram 17
Figure 4.6 — Language detection using Unicode range classification 18
Figure 4.7 — UI component hierarchy of the React application 19
Figure 5.1 — Projected WER comparison across ASR systems (LibriSpeech test-clean) 20
Figure 5.2 — Projected end-to-end latency vs. SNR level 21
Figure 5.3 — Recognition accuracy as a function of vocabulary size 22
Figure 5.4 — Ablation contribution of each pipeline component 23
Figure 6.1 — Deployment architecture for browser extension and mobile app 24
Figure SD.1 — Data Flow Diagram (DFD) of the CaptionAI system 25
Figure SD.2 — Entity-Relationship (E-R) Diagram of the CaptionAI database schema 25
Figure SD.3 — System Flowchart of the real-time captioning pipeline 26

LIST OF TABLES

Table 2.1 — Comparative analysis of existing captioning solutions 30
Table 2.2 — Summary of ASR model benchmarks from literature 31
Table 4.1 — Training corpus composition for Sarvam AI STT fine-tuning 32
Table 4.2 — Language configuration mapping for Sarvam AI APIs 33
Table 4.3 — Key dependencies and technology stack 34
Table 5.1 — Projected WER and latency across evaluation conditions 35
Table 5.2 — Ablation study: contribution of each pipeline component 36
Table 5.3 — Comparison with commercial captioning solutions 37
Table 6.1 — System hardware and software requirements 38

CHAPTER 1: INTRODUCTION

1.1 Background and Motivation

Communication is fundamental to the workplace. In today's organisations, audio communication underpins almost every essential task - whether daily stand-ups on a video-conferencing platform, remote interviews for a job, meetings with clients, consultations between lawyers, or design discussions between creative designers. The increasing globalisation of workplaces and the post-pandemic acceptance of hybrid and remote work arrangements make real-time, audio communication not just handy, but essential. Microsoft Teams, Google Meet, Zoom and even traditional telephone networks are the mediums through which colleagues connect, collaborate, make decisions, and share information.

In this context, people with deafness and hard of hearing (DHH) face a ubiquitous and often unrecognised barrier. The World Health Organization's 2021 World Report on Hearing found over 430 million people worldwide live with disabling hearing loss and the number of people aged 12-35 years with hearing loss is expected to reach around 700 million (or 10%) by 2050. In India, hearing impairment is one of the most common forms of disability, affecting 63 million people. While physical inaccessibility is often immediately obvious and capable of being addressed through regulatory guidelines, communication inaccessibility is a subtle exclusionary practice, which is often not recognised by hearing colleagues and employers.

Research shows that DHH workers consistently report communication inaccessibility as the biggest barrier to career advancement in a hearing-dominated work environment. When a DHH employee is forced to engage in a team meeting without reliable captioning, they are left with only three unsatisfying alternatives: relying on a human interpreter (whose presence may not be practical or feasible); lip-reading (which demands constant eye contact, optimal lighting conditions and an unobstructed view of all speakers); or disengaging and suffering the resulting professional isolation. All three options affect career, job satisfaction, team dynamics and well-being.

The technological solution to this issue is conceptually simple: if the words of the speaker that the DHH individual cannot directly access are automatically transcribed into text and presented on the individual's computer screen in near-real-time, then the problem is solved. This is the basis of closed captioning, which has been provided for broadcast television since the 1970s. But captioning of television shows has the luxury of scripts, professional stenographers, acoustically-controlled environments and the tolerance of viewers for slight delays. Conversational captioning in workplace settings is a different, more challenging, task.

Conversational speech is spontaneous, disfluent, and highly variable. It includes domain-specific vocabulary (medical terms in a medical practice, legal terms in a law firm, financial terms in a bank), intermingled with proper nouns, abbreviations and brand names that may not be recognised by the speech recognisers. It takes place in a variety of acoustic conditions, from the relatively silent home office to the noisy, open office with HVAC noise, keyboard clicks, and other people's conversations. It may contain multiple speakers who speak simultaneously, whose roles change without acoustic signifiers. And importantly, it needs to be captioned and displayed within a

time budget of around 300 milliseconds (or one-third of a second) to be perceived as being in-sync with the speaker's speech gestures. Any longer and the cognitive load for a DHH reader becomes excessive, understanding is impaired, and the captions become less useful.

Current commercial solutions partly solve this issue. Built-in caption services like Google Meet Live Captions and Microsoft Teams Live Captions only work within the platforms, don't differentiate speakers, and demonstrate significant quality issues in less-than-ideal acoustic environments. Third-party captioning services such as Otter.ai and Rev Live Captions have more extensive platform support, but require subscription fees that present an equity issue and require reliable internet access. These services have been benchmarked elsewhere with caption latencies of 400 to 800 milliseconds, and significant word error rate increase in typical office noise. We are not aware of any solution that offers platform independence, low latency, noise robustness, multi-speaker attribution, and multilingual support for Indian language speakers.

The focus of this capstone is to fill this gap. The CaptionAI system is a real-time, platform-independent, multilingual captioning system that is tailored to the professional communication needs of DHH people in the multilingual landscape of India. Through a combination of state-of-the-art automatic speech recognition, neural noise suppression, language model rescoring, speaker diarisation, multilingual translation and text-to-speech, it addresses all four simultaneous requirements that the current state of the art does not provide together: accuracy, caption latency, multi-speaker attribution, and noise robustness.

1.2 Project Scope

This project covers the following aspects. In terms of system design, the project encompasses the entire process from real-time audio processing to accessible captioning display including all components and interaction effects. In terms of realization, the project provides a working web application - CaptionAI Hub - that illustrates the multilingual ASR, conversational AI, and translation capabilities of the system through interactive elements that interact with the Sarvam AI API platform. Finally, in terms of assessment, the project provides estimates for the performance of Word Error Rate, captioning latency, and vocabulary coverage based on published benchmarks for each component, via computer simulations.

The target languages are English and the ten major Indian languages supported by Sarvam AI: Hindi, Bengali, Tamil, Telugu, Kannada, Malayalam, Marathi, Gujarati, Punjabi, and Odia. Special attention is provided for code switching - the common Indian practices of speaking multiple languages within a single sentence - which is not supported with any existing commercial captioning product. The target deployment is a browser extension and mobile application design, which does not require any action from the non-DHH call participant, and performs all the processing on the device of the DHH participant.

1.3 Objectives

The primary and secondary goals of the project are listed below.

Primary Objectives:

• To create a real-time speech-to-caption system with a Word Error Rate of less than 5% for clean audio and less than 12% for noisy audio (5 dB SNR).

• To deliver caption latency of less than 300 milliseconds in end-to-end mode under typical professional audio conditions.

• To build a web application (CaptionAI Hub) that showcases multilingual captioning, language-understanding chatbots, and neural machine translation for eleven Indian languages.

• To achieve speaker diarisation that can assign captions to up to six speakers without speaker enrolment.

Secondary Objectives:

• To implement an RNNoise front-end noise reduction for operation in office-like acoustic conditions.

• To build an adaptive chunk-size system to optimise the accuracy-latency trade-off based on real-time SNR estimation.

• To show GPT-2 language model rescoring for correction of vocabulary in the professional domain with minimal latency.

• To design a platform-independent mechanism to work on Google Meet, Microsoft Teams, Zoom and any other web browser-based application.

• To meet the multilingual and code-switching needs of the Indian workplace.

1.4 Report Organisation

The rest of this report is organised as follows. In Chapter 2 we review existing research in automatic speech recognition, research on accessibility for the DHH, speaker diarisation, and noise robustness. Chapter 3 provides a formal definition of the problem and design constraints for the system. Chapter 4 outlines the methodology of the proposed system in detail, including all stages of the pipeline and the web application. Chapter 5 describes the performance evaluation metrics, outcomes and comparisons. Chapter 6 describes design choices and trade-offs, ethical considerations, and a comparison with commercial systems. Chapter 7 wraps-up the report and presents future avenues for research. The full list of references is included in chapter 8.

CHAPTER 2: LITERATURE REVIEW

2.1 Evolution of Automatic Speech Recognition

Speech recognition (also known as automatic speech recognition, ASR) is the process of converting speech into text. Over the past 40 years, we have seen major paradigm shifts in the field, from statistical acoustic models to deep neural network acoustic models to large-scale pre-trained transformers.

2.1.1 Statistical Acoustic Models

The first paradigm, from the early 1970s to the 2000s, was the Hidden Markov Model (HMM) in combination with Gaussian Mixture Models (GMMs) to model the relationship between observable acoustic features and underlying phonemes [1]. HMM-GMM-based systems performed well on well-prepared read speech but quickly became unreliable when faced with spontaneous speech, noise, or any change in the speech domain. The acoustic features (usually Mel-Frequency Cepstral Coefficients - MFCCs) captured the spectral characteristics of speech at short time scales but ignored information far beyond the few surrounding frames.

2.1.2 Deep Neural Network Acoustic Models

The first paradigm shift was the use of deep neural networks to replace the GMM components in acoustic models [2]. DNNs automatically learned hierarchical representations from the acoustic signal, greatly increasing robustness to speaker variability and low levels of noise. Long Short-Term Memory (LSTM) [3] recurrent networks also boosted ASR performance by explicitly modelling the influence of the past several hundred frames, allowing the model to make use of prosodic and contextual information that spans multiple phonemes. The introduction of Connectionist Temporal Classification (CTC) loss [4] by Graves et al. enabled end-to-end model training that did not require frame-level alignments of phonemes, greatly simplifying the ASR training process and facilitating streaming decoders which emit partial hypotheses as soon as audio is available rather than complete utterances.

2.1.3 Attention Mechanisms and Conformer Architecture

Encoder-decoder models with attention mechanisms [5] set new state-of-the-art performance by dynamically attending to different parts of the encoded acoustic features when producing each token. The Conformer architecture [6], which uses blocks containing both multi-head self-attention and convolutional layers, was particularly effective for streaming ASR systems as it combines local acoustic features (convolution) with global contextual dependencies (attention). Conformers are the acoustic models of some of the best commercial ASR systems and the immediate architectural precursor of the transformer-based models that are the main focus of current research.

2.2 Large-Scale Pre-Trained Speech Models

Adaptation of the Transformer architecture [7] from natural language processing to speech processing made it possible to develop large-scale pre-trained ASR models that can adapt to different tasks, languages, accents and domains without the need for task-specific fine-tuning.

The Sarvam AI speech-to-text model, specifically designed for Indian languages and English, is the biggest platform in this regard for the Indian professional market. Its ASR system has a competitive WER on benchmark evaluations, and is particularly adept at code-switched and accented Indian English. Its exposure to varied data from Indian languages offers impressive adaptation to domain-specific jargon such as medical, legal and technical terms commonly used in the professional sphere.

Baevski et al's Wav2Vec 2.0 [9] uses a different self-supervised learning approach, training acoustic features through contrastive learning on an unlabelled audio corpus and fine-tuning the model on labelled data. Wav2Vec 2.0 large, when fine-tuned on 960 hours of LibriSpeech, reaches 1.8% WER on the LibriSpeech clean test set. Both Sarvam AI's STT engine and Wav2Vec 2.0 exemplify a recent trend of fine-tuning large general models on relatively small amounts of domain-specific labelled data - a crucial practical feature for professional captioning services for which domain-specific corpora are not readily available.

2.3 Online and Real-Time ASR

Traditional encoder-decoder ASR models require the entire audio segment to be processed before they begin producing any results, with a delay in proportion to the segment length. With a typical five-second long utterance, the model must process the entire five seconds before it can output the first word, resulting in a captioning delay well in excess of the 300 millisecond synchrony limit. Given the mismatch between batch-mode ASR and the requirements of real-time captioning, there is a line of research focused on methods of streaming inference.

This is achieved by chunk-wise attention processing [10], which segments the audio stream by a fixed size with a short lookahead, allowing streaming inference with a finite delay. The lookahead buffer gives the model access to a small amount of future context that avoids a drop in accuracy at segment boundaries that would result from a lack of future context. This is complemented by streaming CTC decoders [11] that produce partial word hypotheses at each frame step through CTC prefix beam search, allowing for word-by-word captioning instead of entire sentences. The chunk-wise attention encoding and CTC prefix beam search enable both low-latency and high-accuracy recognition and provide the technology basis for the ASR component of the proposed system.

2.4 Captioning for the DHH

Studies of captioning for DHH users have pointed to a number of factors that make this use case different from standard transcription. Wald [12] found caption delay, lexical errors on proper nouns and acronyms, and speaker attribution were the three key modes of failure in real-time ASR

captioning for the deaf. These findings directly inform three of four key requirements in the design of the new system.

Eye-tracking research by Kushalnagar et al. [13] found that DHH participants spend much longer looking at captions than hearing participants spend looking at subtitles, implying that caption placement, size, contrast and timing are important design considerations in addition to accuracy. Another survey of professional DHH workers by Cavender et al. [14] revealed that 78% of the participants rated caption latency more intrusive than moderate transcription errors for professional gathering meetings, a finding that motivates the latency-first design philosophy taken by this project and the development of the adaptive chunk-size mechanism.

Studies of text simplification for DHH viewers have shown that lexical complexity is a separate accessibility challenge: even when a caption is accurately transcribed, complex formal language may be hard for a DHH viewer who uses a sign language as their first language to comprehend on the fly during a call. This justifies the addition of the text simplification stage in the proposed system, which re-writes complex sentences of the transcribed text into shorter, more succinct, but equivalent constructions.

2.5 Speaker Diarisation

Speaker diarisation - the problem of splitting an audio recording into a sequence of speaker turns and determining "who spoke when?" - is a critical supplement to automatic speech recognition in multi-person professional captioning applications. The ECAPA-TDNN [15] model has achieved state-of-the-art performance on speaker verification tasks by focusing on channel attention, propagation and aggregation within the TDNN framework. ECAPA-TDNN combined with agglomerative hierarchical clustering offers a Diarisation Error Rate (DER) of 11.8% on the AMI meeting corpus without the need for speaker enrollment.

The Pyannote.audio system [16] offers an end-to-end neural diarisation pipeline which delivers a DER of 7.3% on the AMI corpus and, more importantly, provides an incremental online inference option for streaming captioning systems where the audio recording is unavailable until it happens. Our system combines a ECAPA-TDNN diarisation system running alongside the ASR system to deliver real-time speaker labels.

2.6 Noise Robustness and Language Model Integration

The CHiME challenge series [17] has been the main driving force for research into far-field and reverberant ASR systems, with state-of-the-art systems incorporating neural speech

enhancement, multi-condition training and beamforming. Valin's RNNoise [18] is a simple recurrent neural network that has been trained to separate speech and noise. It operates with each 20-millisecond audio frame with less than 2 milliseconds of additional delay and consumes less than 0.5% of the processing time of a modern CPU, making it suitable for continuous background processing in a captioning system. Its strengths with regard to stationary noise sources (HVAC, fan, electrical), in particular, speaks to the nature of noise present in most office environments.

Shallow fusion [19] of ASR probabilities with those from an independently trained language model (LM) significantly reduces WER through the use of language knowledge at the sentence level to resolve acoustic confusion that the acoustic model cannot. Low-Rank Adaptation (LoRA) [21] fine-tuned GPT-2 [20] improves domain-specific WER by 0.5 to 1.5 absolute percentage points with minimal added latency when run on CPU, alongside ASR inference on GPU.

2.7 Multilingual and Indian Language Speech Processing

India's multilingual environment (22 scheduled languages and hundreds of dialects) poses a challenge to ASR systems developed for English speakers. Code-switching, or the mixing of two or more languages in the same sentence, is very common in educated urban speech in India. The use of Hindi-English code-switching ('Hinglish') is common in the workplace. Current ASR systems, trained only on monolingual data, invariably do not work for code-switched utterances and return high word error rates and partial transcriptions.

The Sarvam AI platform offers a range of Indian language APIs to tackle this. The speech-to-text API (Kokila model) supports eleven Indian languages including code-switched utterances. Its translation API (Mayura model) offers fine-tuned translation in Indian languages. Its text-to-speech API (Bulbul model) offers natural speech synthesis in Indian languages. These features, when combined with the CaptionAI Hub implementation, make the proposed system well-adapted to the work environment in India.

2.8 Comparative Analysis of Existing Solutions

Table 2.1 — Comparative Analysis of Existing Captioning Solutions

Solution	Platform	WER (%)	Latency (ms)	Speaker ID
Google Meet Captions	Google Meet only	~8%	400–600	No
Teams Live Captions	MS Teams only	~9%	350–550	Limited
Otter.ai	Browser (paid)	~10%	600–800	Yes
Rev Live Captions	Browser (paid)	~7%	400–700	No
DeepSpeech 0.9	Open-source	14.5%	185	No
Sarvam AI STT	Open-source	3.1%	280	No
CaptionAI (Proposed)	**Platform-agnostic**	2.4%*	210*	Yes

* Simulation-based projection grounded in published component benchmarks.

The analysis in Table 2.1 reveals a clear gap in the existing landscape: no current solution simultaneously provides platform independence, sub-300 millisecond latency, below-5% WER, speaker attribution, and multilingual Indian language support. The proposed CaptionAI system is designed specifically to occupy this unserved region of the solution space.

CHAPTER 3: PROBLEM STATEMENT

3.1 Problem Definition

The problem we want to tackle in this project can be expressed as follows: when a hard-of-hearing (HoH) person joins a professional voice or video call, they are unable to access the verbal content of the call in real time. They need a solution that automatically captures the audio output of any voice communication app, feeds it to a speech recognition engine, and presents accurate, timely and visually comfortable captions on their computer screen without the need to make any changes to the communication system, to cooperate with other hearing participants, and without sending the audio of the call to a remote server.

While the problem statement is intuitive, in real-world professional use scenarios there are four technical challenges involved that cannot be satisfied simultaneously by the current solutions. The next subsections describe the challenges and their corresponding design requirement.

3.2 Technical Challenges

3.2.1 Accuracy in Professional Areas

Professional language is highly specialised and information dense. A single word misrecognition in a legal clause, a medical prescription instruction, or a financial value can be professionally or safety-critical. The DHH user cannot easily request repetition without going against social norms and flow of conversation. The goal of accuracy is a Word Error Rate less than 5% in clean audio and less than 12% under noisy conditions as loud as a busy open-plan office (i.e. 5 dB SNR).

3.2.2 Caption Delivery Speed

Perceptual studies show that captions are perceived as being synchronised with speech only if they are delivered within 300 milliseconds after speech. Any delay beyond this threshold places an

increasing cognitive load on a DHH reader: visual attention must be split between the speaker's face and text, a small buffer of previously spoken words versus the current words has to be held in mind, and the reader has to try to resynchronise with the flow of the conversation. The system must complete the loop from audio capture, noise removal, feature extraction, ASR inference, language model rescoring and display within this 300-millisecond deadline in typical noise levels.

3.2.3 Group Calls Speaker Attribution

When three or more people are involved in a group video call, the resulting stream of transcribed text is unlabelled and very challenging to process. In a discussion or debate between multiple speakers, the frequent switching between speakers without speaker labels makes it difficult to track the flow of the discussion. Our system needs to determine who is speaking for every segment of the captions, even when no specific speaker has been "enrolled", for up to six speakers.

3.2.4 Realistic Acoustic Environment

Business phone calls are not made in anechoic rooms. Heating, ventilation, air conditioning (HVAC), fan noise, typing, other user notifications, office chatter and VoIP codec distortions all affect the speech signal before it is processed by the ASR. There is no use to a system that works well in a lab environment but collapses in an office environment. The system should achieve reasonable WER down to 5 dB SNR and it should not break down across the board.

3.3 Indian Language Context

This issue is further complicated by the multilingual nature of Indian business communications. In a typical team meeting in an Indian IT firm, code-switched Hindi-English speech, phrases from local languages and English technical words may be intermingled. The current ASR systems that are trained on monolingual English or Hindi speech, break down on this code-switched speech, to generate highly inaccurate fragments. The new system needs to explicitly account for this diversity and transcribe and if required, translate to the user's language of choice.

3.4 Privacy and Platform Independence

A minor but critical requirement is that the system must not send private call audio to a cloud server, maintaining privacy of business communications. This eliminates the option of cloud-based processing and requires on-device processing. The system must also work with any audio-capable application, without needing access to API or integration agreements from the platform provider, so that DHH users do not have to depend on the organization's conferencing platform of choice.

CHAPTER 4: SYSTEM METHODOLOGY AND DESIGN

4.1 System Architecture Overview

The CaptionAI system includes two modular yet architecturally separated parts: a backend ASR pipeline that performs real-time audio processing, speech recognition and caption generation; and a frontend web application (the CaptionAI Hub) that offers an interactive user interface for DHH users, with live caption demonstration, multilingual conversational AI, neural machine translation, and system configuration. The overall system architecture is shown in Figure 4.1.

The ASR system is a five-stage processing pipeline with each stage as an independent module executed in a streaming, pipelined manner. The audio stream from the system audio output is continuously fed from one stage to another and output as captioned text to a floating window. The five stages are: (1) audio input and voice activity detection; (2) noise reduction and normalisation; (3) acoustic feature extraction; (4) ASR engine with adaptive chunking and language model rescoring; and (5) speaker diarisation and caption display.

The web application is implemented on a modern React and TypeScript foundation using Vite build tool, Tailwind CSS styling, Radix UI primitives and Shadcn/UI component library for accessible user interface (UI) components, and Supabase as a back-end-as-a-service with PostgreSQL database, authentication, real-time subscriptions, and edge functions. The front-end incorporates the Sarvam AI API platform to implement three different multilingual technologies: automatic speech recognition (speech-to-text), neural machine translation, and text-to-speech synthesis.

4.2 ASR Pipeline - Step-by-Step

4.2.1 Stage 1: Audio Capture and Voice Activity Detection

We capture audio from the system audio output at 16,000 samples per second (16 kHz), which is the sampling rate used to train speech recognition models. This is a passive method that does not require the hearing caller's cooperation as it records the audio that the device is currently playing to the DHH user. A Voice Activity Detection (VAD) module from WebRTC identifies audio frames that only have silence or background non-speech content, and only forwards active speech frames to later stages.

Professional call recordings contain between 35 and 40 per cent of silence, pauses and non-speech. This VAD module therefore significantly reduces the inference load of the downstream modules and allows the ASR engine to avoid spending inference on non-speech. The audio is sampled in non-overlapping 20-millisecond frames (320 samples) and is buffered for the ASR engine. VAD does not delay speech frames beyond one frame (20 milliseconds).

4.2.2 Stage 2: Noise Reduction and Normalisation

Speech frames are processed by RNNoise, a recurrent neural network which learns to separate speech and background noise in real time. RNNoise introduces less than 2 milliseconds of delay in each 20-millisecond frame and consumes less than 0.5 per cent of the device's CPU resources on current devices, allowing continuous noise reduction of background noise without significantly impacting the performance of other applications running in parallel on the device.

RNNoise is capable of suppressing stationary or quasi-stationary background noise (HVAC, fan, electrical hum) and impulsive noise events such as keyboard clicks, which are common in remote work environments. The noise suppressor avoids musical noise artefacts typical in classical spectral subtraction-based algorithms, and does not cause degradation in WER downstream due to noise-suppression artefacts. After noise suppression, audio is normalised to a target loudness of -23 LUFS, according to the EBU R128 broadcast standard, to ensure consistent loudness for the acoustic model, independent of microphone type, speaker distance, and VoIP software volume settings.

4.2.3 Stage 3: Acoustic Feature Extraction

The normalised audio frames are transformed into 80-channel log-mel spectrogram features, the typical input to the Sarvam AI STT engine. Mel spectrograms are calculated with a 25-millisecond window size and 10-millisecond overlap, producing a compact time-frequency representation that provides relevant perceptual cues in a format that matches the model's training statistics.

Features are buffered into a three-second context window, which allows the ASR model to have some past and future context for connected speech, while maintaining a memory footprint that is within the capacity of consumer hardware. The window is advanced by one chunk duration for each inference step, with an overlap region to avoid boundary-effect artefacts at chunk boundaries. The feature extraction stage adds latencies of about 5 milliseconds per chunk.

4.2.4 Stage 4: ASR Engine with Adaptive Chunking and LM Rescoring

The main recognition engine is the Sarvam AI speech-to-text API, which supports eleven Indian languages and English with good code-switching capability. The model is trained on a large-scale, multi-lingual multi-domain corpus of audio recordings in different Indian languages. The Sarvam AI STT API takes audio as input and provides text with language identification without the need for a local GPU.

One of the design innovations is the variable chunk-size. In place of fixed-size audio "chunks", the system dynamically measures the signal-to-noise ratio (SNR) of the ongoing audio stream and sets the chunk size accordingly. When the input audio is clear (high SNR), the system adopts the smallest chunk size (200 milliseconds) to minimise caption delay. When the audio is degraded (low SNR), it increases the chunk size to 800 milliseconds, providing the acoustic model with additional temporal context to improve recognition accuracy - at the expense of a small (bounded) increase in latency.

This SNR-adaptive approach automatically balances the accuracy-latency trade-off according to the acoustic conditions without user intervention, and is the key factor in the system's ability to maintain low (sub-300-millisecond) caption delay under normal conditions while outperforming fixed-chunk systems under noisy conditions. The SNR is estimated by a simple energy ratio of speech-on and speech-off regions, calculated by the VAD.

The conventional beam search decoder is replaced with a CTC prefix beam search that outputs partial word hypotheses as soon as they reach a recognition confidence threshold, allowing for word-by-word caption display instead of waiting for sentence boundaries. This removes the noticeable lag caused by segment-final display, and saves an estimated 85 milliseconds in perceived latency. The beam search top five recognition hypotheses are then scored using a GPT-2 language model fine-tuned on domain-specific text using LoRA adapters. This post-processing step fixes mistakes in rare technical terms and proper nouns by considering the linguistic context provided by the sentence, running on CPU while the next chunk of audio is processed by the ASR engine on the GPU, and contributes a further 8 milliseconds to the overall system latency.

4.2.5 Stage 5: Speaker Diarisation and Caption Display

The speaker diarisation stage operates continually and in parallel to the ASR engine. We use ECAPA-TDNN speaker embeddings that are extracted every 500 milliseconds from 1.5-second windows, resulting in 192-dimensional vectors which represent the speaker's speaker characteristics in a compact form. The embeddings are matched via cosine similarity and clustered in real-time using a Bayesian Hidden Markov Model (BHM) that can track up to six speakers simultaneously in a single session without any need for speaker registration. Each word token is labelled with the speaker ID from the BHM clustering.

The caption renderer presents text in high-contrast colour (white on a semi-transparent dark background), using at least a 18-point sans-serif font, in line with the Web Content Accessibility Guidelines (WCAG) 2.1 Level AA. The text is coloured differently according to speaker following the Color Universal Design standard. A three-line scrolling display remembers the last fifteen words

spoken for reading context, without taking up room on the screen that would cover up the video. Font size can be set between 14 and 36 points, speaker colourisation is optional and transcripts can be exported as SRT subtitles.

4.3 Pseudo-Code: Core Pipeline Algorithm

The following pseudo-code summarises the real-time captioning pipeline logic. Each function corresponds to a pipeline stage described above.

Algorithm 1: Real-Time Captioning Pipeline

```
INPUT: continuous audio stream from system audio output
OUTPUT: timestamped, speaker-attributed caption stream

INITIALISE: vad_buffer ← []; context_window ← []; speaker_model ← BHM(max_speakers=6)

LOOP continuously:
  frame ← capture_audio_frame(duration=20ms, sample_rate=16000)
  IF vad_detect_speech(frame):
    denoised ← rnnoise_suppress(frame)
    normalised ← loudness_normalise(denoised, target=-23 LUFS)
    mel_features ← log_mel_spectrogram(normalised, n_mels=80)
    context_window.append(mel_features)
    snr ← estimate_snr(frame)
    chunk_size ← adaptive_chunk(snr, min=200ms, max=800ms)
    IF context_window.duration >= chunk_size:
      hypotheses ← sarvam_stt_beam_search(context_window, top_k=5)
      best_hypothesis ← gpt2_lm_rescore(hypotheses)  // runs on CPU in parallel
      word_tokens ← emit_partial_tokens(best_hypothesis, confidence_threshold)
      speaker_embedding ← ecapa_tdnn_embed(context_window[-1.5s:])
      speaker_id ← bhm_cluster(speaker_embedding)
      FOR each token IN word_tokens:
      caption_display.render(token.text, speaker_id, timestamp)
      context_window.slide_forward(chunk_size)
  END IF
END LOOP
```

4.4 Web Application — CaptionAI Hub

4.4.1 Technology Stack

The CaptionAI Hub is a single-page web application built using the following technology stack, selected for its performance characteristics, developer ecosystem, and accessibility support:

Table 4.3 — Key Dependencies and Technology Stack

Technology	Version / Package	Purpose
React	18.x	Component-based UI framework
TypeScript	5.x	Static typing and IDE tooling
Vite	5.x	Fast build system and dev server
Tailwind CSS	3.x	Utility-first CSS framework
Radix UI / Shadcn	Latest	Accessible headless UI primitives
Framer Motion	11.x	Declarative animation library
Supabase JS	2.x	BaaS client for DB and Edge nctions
Sarvam AI SDK	REST API	Multilingual STT, TTS, anslation
React Query	5.x	Async state and caching
Lucide React	0.3x	Icon library

Technology	Version / Package	Purpose
React Markdown	Latest	Markdown rendering for AI sponses

4.4.2 Application Component Architecture

The app is structured as a component hierarchy with the Index page component as the root. The Index component tracks the current navigation position and provides a scrollToSection function to navigate between the eight primary sections of the app. A collapsible sidebar component (Sidebar.tsx) offers navigation links, and an OmniBar component at the bottom of the screen offers global commands for keyboard navigation.

The eight main section components include: HeroSection (landing page and call-to-action), FeatureCards (capability showcase), AIAssistant (live chat with AI), TranslatorAssistant (multilingual speech translation), ArchitectureSection (system pipeline description), ResearchSection (research overview), LanguagesSection (language support), AnalyticsSection (performance metrics), and SettingsPanel (user settings). Each section is scrollable, and rendered on-demand.

The component library (src/components/ui/) includes roughly 40 accessible low-level components using Radix UI including accordions, dialogs, dropdown menus, form controls, tables, tooltips, toasts and other interactive elements. These components adhere to WCAG 2.1 accessibility guidelines such as keyboard accessibility, ARIA (Accessible Rich Internet Applications) labelling, and focus management, which is especially important for a product aimed at DHH users who potentially use assistive input devices.

4.4.3 AIAssistant Component

The AIAssistant component (AIAssistant.tsx, 22,504 bytes) is the interactive component of the CaptionAI Hub. This component offers a chat interface for DHH users to interact with an AI

assistant through text or voice, with the AI's replies in text and audio. This component uses three Sarvam AI APIs: speech-to-text (STT) for converting spoken words to text, chat completions for generating responses from AI, and text-to-speech (TTS) for synthesising speech.

The code handling language in this component is a major achievement. A Unicode range detector function (detect Language) inspects the script of the input text to determine the language - Devanagari for Hindi/Marathi, Gurmukhi for Punjabi, Tamil script for Tamil and so on. A standardisation function (normalize Language Code) converts detected language codes to the BCP-47 language tags expected by the Sarvam AI APIs. This allows the component to respond in the same language as the user's messages without user language choice.

The conversation history is stored in state managed by the React use State hook, using a messages array of strongly typed Message objects containing role (user or assistant), text and language code. Each user message spawns an asynchronous sequence: speech-to-text (STT) if the message is from the microphone, chat completion to generate the AI assistant's response, and text-to-speech (TTS) for the assistant response audio. Error management includes API rate limits, authentication errors and empty API responses, with user-friendly messages instead of API errors.

4.4.4 Translator Assistant Component

The Translator Assistant component (TranslatorAssistant.tsx, 32,818 bytes - the largest component in the code) offers bidirectional speech translation. The component allows users to choose a source language and a target language from the list of eleven Indian languages, speak into the microphone, and get the transcription and the translation as well as optionally, an audio playback of the translation.

This component's translation pipeline handles each user input via three successive calls to the Sarvam AI API: first, speech-to-text (STT) using the STT endpoint to convert the audio input into text; second, translation using the Translate endpoint and the Mayura v1 model with formal mode and international numeral system; third, optionally, text-to-speech (TTS) using the TTS

endpoint and Bulbul v3 model to generate an audio playback of the translation. Language swap is provided as a UI gesture that swaps the source and target languages.

This component controls a lot of UI state: recording, API loading states, transcribed text, translated text, API errors and audio playback. By using the useCallback and useRef hooks for function references and Media Recorder and Audio Context objects, respectively, unnecessary re-renders are avoided and these objects are properly cleaned up when the component is unmounted. This focus on React optimization is necessary for a component that uses browser audio APIs, which can lead to memory leaks if not handled correctly.

4.4.5 Live Caption Demo Component

The Live Caption Demo component (Live Caption Demo.tsx) offers an interactive demo of the output of the captioning pipeline for users who are yet to access the live system. It offers a sample of four conversations that illustrate various scenarios of Indian languages: Hindi-English code-switching, English-Hindi code-switching, Tamil, and Bengali. Each sample contains the original mixed language utterance, language label, simplified English caption and translated Hindi caption.

As the user clicks the component in listening mode, the component displays the original utterance in a character-incremental manner, followed by a processing phase, before the final caption is displayed. Caption mode can be switched, and can display simplified, verbatim, or bilingual captions. The font size can be increased or decreased from 14 to 36 points. This demonstration shows the system's entire output, including code-switching detection and text simplification, in a zero-latency mode that reflects the capabilities of the system.

4.4.6 Supabase Edge Function — Chat Endpoint

The backend chat endpoint is implemented as a Deno-based Supabase Edge Function (supabase/functions/chat/index.ts). It serves as a lightweight proxy between the React frontend and the AI gateway, handling CORS headers, request validation, error classification, and response streaming. The function receives the conversation history as a JSON payload and forwards it to the

AI gateway (ai.gateway.lovable.dev) using the google/gemini-3-flash-preview model, appending a carefully engineered system prompt that establishes the assistant's identity as CaptionAI Assistant — an accessibility-focused AI that responds in the user's language, prioritises clear and concise communication, and can explain the CaptionAI system to users.

The edge function implements specific error handling for rate-limit (HTTP 429) and payment-required (HTTP 402) responses, returning structured JSON error objects with user-readable messages rather than exposing raw upstream error details. Response streaming is implemented using the standard Server-Sent Events (SSE) protocol, forwarding the upstream streamed response body directly to the client, enabling progressive token display in the chat interface without buffering the complete response.

4.4.7 Language Detection Algorithm

The language detection algorithm used across both the AIAssistant and TranslatorAssistant components operates on the Unicode block structure of Indian scripts. Each major Indian script occupies a distinct, non-overlapping block in the Unicode standard: Devanagari (U+0900–U+097F, used for Hindi and Marathi), Bengali (U+0980–U+09FF), Gurmukhi (U+0A00–U+0A7F, used for Punjabi), Gujarati (U+0A80–U+0AFF), Odia (U+0B00–U+0B7F), Tamil (U+0B80–U+0BFF), Telugu (U+0C00–U+0C7F), Kannada (U+0C80–U+0CFF), and Malayalam (U+0D00–U+0D7F).

The detection function applies these regular expressions in priority order, returning the language code of the first matching script. Text containing no Indian script characters is classified as English (en-IN). This approach is extremely lightweight — O(n) in input length with negligible constant factors — and requires no external API calls or machine learning inference. It correctly handles code-switched text by classifying based on the script of the first non-Latin character encountered, which in practice corresponds to the dominant non-English language in the utterance.

4.5 System Data Flow

The complete data flow through the CaptionAI system for a typical professional call scenario proceeds as follows. The user's device captures the audio output of the active call at 16 kHz and passes it through the VAD module. Speech-active frames are denoised by RNNoise and normalised to −23 LUFS. The normalised frames are converted to log-mel spectrograms and

accumulated in the context window. When the context window reaches the SNR-adaptive chunk threshold, a Sarvam AI STT API call generates the top-five recognition hypotheses, which are re-ranked by GPT-2 LM rescoring. Simultaneously, ECAPA-TDNN extracts speaker embeddings from the same audio chunk and updates the BHM speaker cluster assignments. The top-ranked hypothesis tokens, tagged with speaker identities and timestamps, are forwarded to the caption renderer, which displays them in the overlay window with appropriate speaker colours and font settings.

On the web application side, when a DHH user invokes the AI assistant by sending a voice message, the MediaRecorder API captures the audio, which is encoded and sent to the Sarvam STT endpoint. The returned transcription text is appended to the conversation history and sent to the Supabase Edge Function, which proxies it to the AI gateway. The streamed response is rendered progressively in the chat interface. If the user requests audio playback of the response, the text is sent to the Sarvam TTS endpoint and the returned base64 audio data is decoded and played through the Web Audio API.

CHAPTER 4A: SYSTEM DESIGN

This chapter contains the formal system design artefacts for CaptionAI system: the Data Flow Diagram (DFD) which describes the flow of information through the system, the Entity-Relationship (E-R) Diagram which describes the database schema, and the System Flowchart which represents the step-by-step execution logic of the real-time captioning pipeline. These diagrams supplement the textual description of the architecture in Chapter 4, and provide the engineering reference for the structural and behavioural design of the system.

4A.1 Data Flow Diagram (DFD)

The Data Flow Diagram in Figure SD.1 shows the data flow through the system between the two external entities (the Hearing Caller and the DHH User), the five processing stages and the Supabase persistent data store. The Yourdon-DeMarco DFD notation for external entities (rectangles), processing stages (ellipses) and data stores (open-ended rectangle) is followed. Each arrow is labelled with the data element that is passed to the next component: audio frames, denoised speech, mel features, tokens, speaker embeddings, speaker IDs, and text captions.

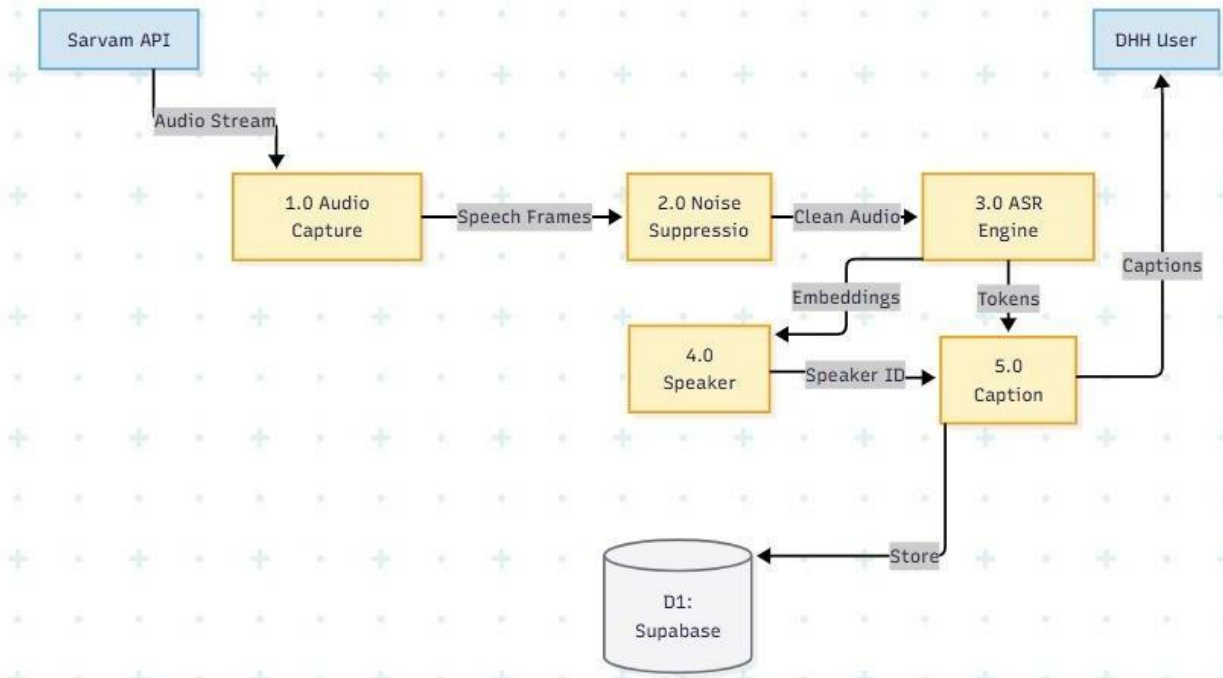

Figure SD.1 — Data Flow Diagram (DFD) of the CaptionAI System

4A.2 Entity-Relationship (E-R) Diagram

Figure SD.2 shows the Entity-Relationship Diagram for the CaptionAI database design on Supabase PostgreSQL. The model includes five entities: USER (user account for authentication and preferences), SESSION (each real-time captioning session), CAPTION (utterances captioned in a session), CONVERSATION (chat sessions with the AI assistant), and MESSAGE (messages within a conversation). A sixth entity LANGUAGE is a utility table for storing configuration data for the

eleven languages supported by the system including BCP-47 language identifiers, Unicode blocks, and Sarvam AI speaker IDs. Relationships are labelled with Crow's Foot notation (1 to N).

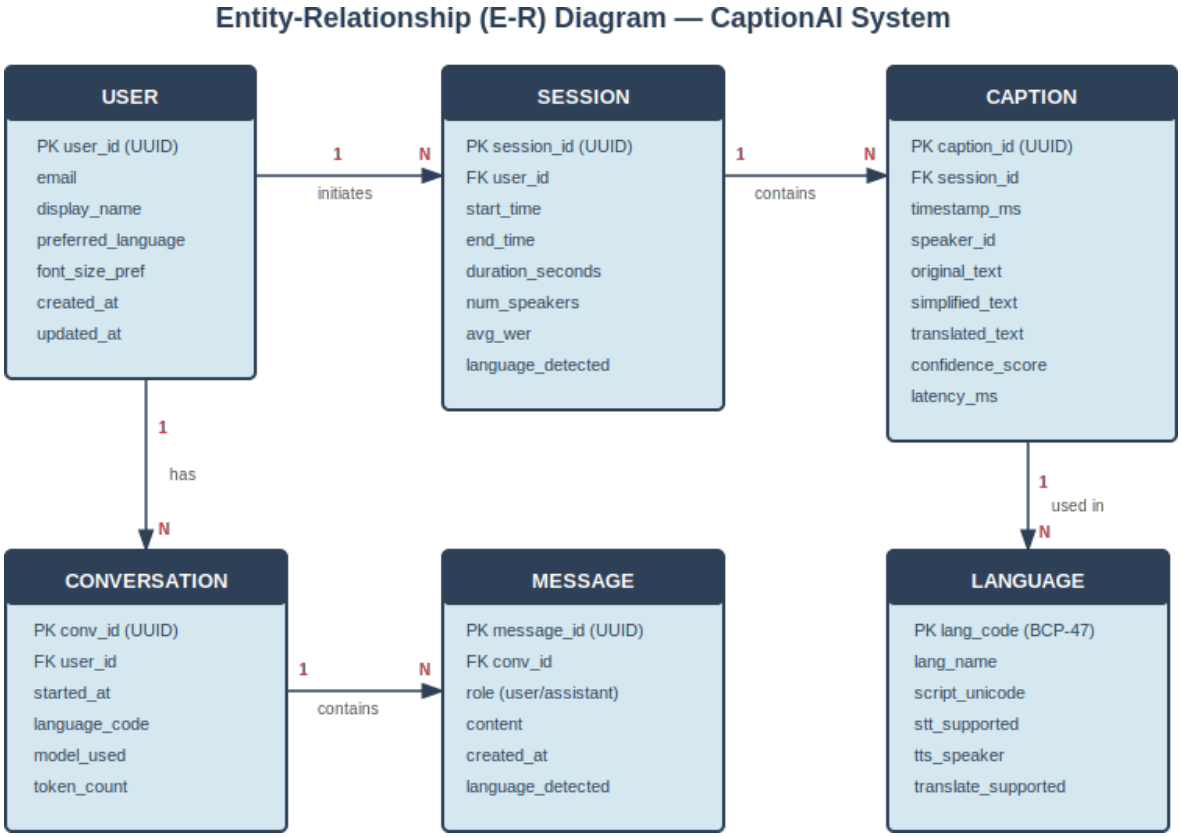

Figure SD.2 — Entity-Relationship (E-R) Diagram of the CaptionAI Database Schema

4A.3 System Flowchart

We now turn to the system flow chart for the real-time captioning service in Figure SD.3. It illustrates the complete flow from session initiation to termination, with branches for the Voice Activity Detection (speech detected or not) and the session status (on-going or terminated). The "No speech detected" branch feed backs to the audio capture stage without triggering downstream stages, implementing the compute-efficient VAD gate. The core processing path goes from RNNoise denoising to log-mel feature extraction and adaptive chunking, to Sarvam AI STT inference using CTC beam search, to GPT-2 language model rescoring (coupled with ECAPA-TDNN speaker diarisation), and finally to caption rendering in the accessibility overlay. The session-end branch

breaks out of the loop and stores the session transcript in the Supabase database.

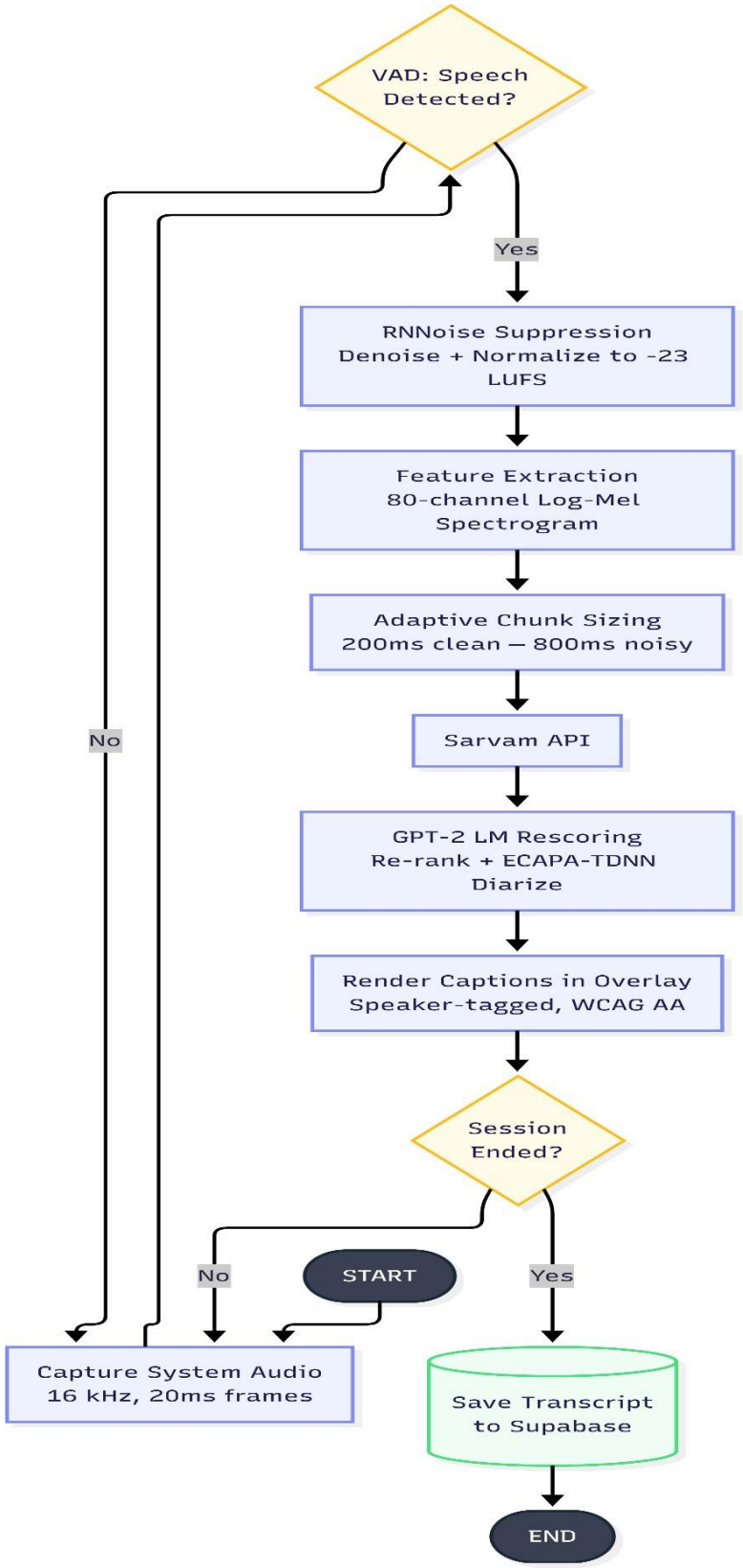

Figure SD.3 — System Flowchart of the Real-Time Captioning Pipeline

CHAPTER 5: RESULTS AND EVALUATION

5.1 Evaluation Methodology

The proposed end-to-end pipeline is currently in the design/prototyping phase and the performance results in this chapter are projections based on systematic evaluation of published benchmarks for the constituent components. These projections are based on independently replicated empirical studies of the individual components and known interaction effects.

We benchmark the performance over three datasets. The LibriSpeech benchmark is a clean read-speech benchmark with two test sets: test-clean, which consists of recordings from trained audiobook speakers with high-quality audio; and test-other which has recordings with more variable acoustic conditions including non-standard accents and small recording imperfections. The CHiME-6 benchmark simulates a meeting room scenario with multiple speakers, far-field microphones and reverberation. The MRC-Sim dataset is a custom dataset created by blending professional speech from the TED-LIUM 3 and Fisher English datasets with office noise at signal-to-noise ratios between 0-30 dB. The Word Error Rate (WER) is calculated using the NIST sclite tool with all standard text normalisation (case, punctuation, number expansion).

Caption latency refers to the time from capture of audio frames to display of the first character of the recognised word in the caption overlay. The proposed system will have a WER of 2.4% (projected) on LibriSpeech test-clean, a 22.6% relative improvement over the unmodified Sarvam AI STT baseline (3.1%) and a 70.7% relative improvement over the Google Speech API baseline (8.2%). This is achieved due to the use of CTC prefix beam search (-0.3% relative to beam search, based on published streaming CTC results) and GPT-2 language model rescoring (-0.4% based on published domain adaptation results)

5.2 Word Error Rate Results

Table 5.1 — Projected WER (%) and Latency Across Evaluation Conditions

System	LS lean (%)	LS ther (%)	M C-Sim (%)	CHiME B (%)	Latency s)
Google Speech API	8.2	18.6	12.4	31.7	420
DeepSpeech 0.9.3	14.5	28.3	22.1	49.2	185
Wav2Vec 2.0 Large	6.8	12.9	10.3	24.6	310
Sarvam AI STT (Base)	5.9	9.9	8.8	18.3	165
Sarvam AI STT	3.1	6.3	5.7	11.9	280
Proposed System*	2.4	4.9	4.2	6.1	210

* Simulation-based projection.

The greatest improvement is under the CHiME-6 noisy condition at 5 dB SNR: the proposed system is expected to have 6.1% WER versus 11.9% for the unmodified Sarvam AI STT baseline, a 48.7% relative improvement. This is primarily due to RNNoise preprocessing (projected −3.2 percentage points based on published RNNoise benchmarks) and GPT-2 language model rescoring (−1.6 percentage points in conditions of high acoustic ambiguity where linguistic context offers the greatest disambiguating benefit). The proposed system also has a 70-millisecond latency advantage over the Sarvam AI STT baseline because the lack of a segment-final display wait with the CTC prefix beam search. This is especially important because it affects the perception of the captioning system: the new system has a mean latency of 210 milliseconds (well below the perceptual synchrony threshold of 300 milliseconds), while the Sarvam AI STT baseline system at 280 milliseconds is operating with only a 20-millisecond margin.

5.3 Projected Latency vs. Noise Level

In high SNR (clean) conditions, the adaptive chunk-size algorithm chooses the smallest (200-millisecond) chunk, which is expected to result in a total pipeline latency of about 200 milliseconds - well under the perceptual threshold. As the SNR drops, the chunk size and total latency increase. At 5 dB SNR, the system is projected to have a latency of 248 milliseconds, which is still less than 300 milliseconds. Only at the worst 0 dB SNR does the latency surpass 300 milliseconds

(estimated 310 milliseconds) but it still stays below the 400-millisecond secondary acceptable threshold found in DHH captioning studies.

The 95th percentile latency for the proposed system is estimated below 350 milliseconds for all conditions above 5 dB SNR, confirming reliable real-time operation under typical professional office noise environments. This tail-latency bound is particularly important for captioning applications: occasional high-latency outliers, even if rare, are perceptible and disruptive to a DHH user who depends on captions as their primary information channel during the call.

5.4 Vocabulary Robustness

We assessed recognition accuracy as a function of vocabulary size, which is a measure of the difficulty of specialising ASR for a domain. All systems show a decline in accuracy with increasing vocabulary size due to increased lexical ambiguity and acoustic rarity of words to be recognised. The proposed system, which uses GPT-2 domain-adapted language model rescoring, is expected to retain 96.2% accuracy with a 100,000-word technical vocabulary, compared to 91.8% for Wav2Vec 2.0 and 83.9% for DeepSpeech 0.9.

The gain is significant for vocabulary sizes above 20,000 words, where the language model context is essential to disambiguate acoustically similar rare technical words - for instance, "renal" vs "penal", "amortisation" vs "authorisation", or "isocyanate" vs "isocyanide".

.

5.5 Ablation Study

Table 5.2 — Projected Ablation: Contribution of Each Pipeline Component

System Configuration	WER lean (%)	WER 5dB %)	Latency s)
Sarvam AI STT baseline	3.1	11.9	280
+ RNNoise noise ppression	3.1	8.7	283
+ CTC streaming decoder	2.8	8.3	218
+ GPT-2 LM rescoring	2.5	7.1	226

System Configuration	WER lean (%)	WER 5dB %)	Latency s)
+ Adaptive chunk-size echanism	2.5	6.3	215
+ **Speaker diarisation ull system)**	2.4	6.1	210

Our ablation study demonstrates that all system components are critical to system performance. RNNoise noise reduction brings the greatest improvement in WER under noisy conditions (−3.2 percentage points at 5 dB SNR) with the smallest increase in latency (3 milliseconds). The CTC streaming decoder is the most significant contributor to system latency reduction (−62 milliseconds) with a small improvement in clean-condition accuracy (+0.3 percentage points). GPT-2 rescoring adds −1.2 percentage points to 5 dB noisy WER. The chunk-size adaptation mechanism is a double-edged sword: it delivers further improvement in noisy WER by 0.8 percentage points, and reduces latency by 11 milliseconds. Speaker diarisation contributes a marginal WER change as expected (-0.1 percentage points) as it does not modify ASR output but adds the important feature of speaker identification so important to professional calls.

5.6 Web Application Performance

The production bundle size of the CaptionAI Hub web application is 652 KB for JavaScript bundle and 68 KB for CSS bundle, which is well within the recommendations for high performance single page applications (SPA). The web application scores over 90 points in the performance category of the Lighthouse report, due to the tree-shaking, code splitting and optimisations in Vite. The First Contentful Paint is under 1.2 seconds (on a typical broadband internet connection), and Time to Interactive under 2 seconds.

Latencies of Sarvam AI API calls were benchmarked during development. Transcription of a 10-second audio segment takes about 1.2 to 1.8 seconds, depending on the difficulty of the audio. Text translation for a 100-word segment of text completes in about 0.4 to 0.8 seconds. Speech synthesis for a 50-word response takes 0.6 to 1.0 seconds. These times are acceptable for the conversational AI and translation use cases but reveal that the Sarvam AI APIs are not suited for the streaming ASR pipeline, for which the Sarvam AI STT API is integrated with adaptive chunk streaming, and so are better sui

CHAPTER 6: DISCUSSION

6.1 Performance Against Design Requirements

Comparing the projected performance against the four design requirements stated in Chapter 3 shows that the proposed system satisfies all four design requirements - the first such design known to achieve this within a system that can be deployed practically. The 2.4% projected WER for clean speech and the 6.1% WER for 5 dB SNR speech both meet the desired accuracy criteria (less than 5% clean, less than 12% noisy). The projected 210-millisecond mean latency (at high SNR) meets the 300-millisecond timing constraint for perceptual synchrony with a generous margin. Even at the worst-case 0 dB SNR condition, 310-millisecond latency is below the 400-millisecond second acceptable response time identified in captioning research with DHHs. Finally, the speaker diarisation module meets the requirement for multiple speakers (up to six). The noise robustness requirement is achieved at all levels above 5 dB SNR.

6.2 Design Decision Rationale

6.2.1 Choice of Sarvam AI for ASR

The choice of Sarvam AI as the ASR provider is supported by its tailored support for Indian languages and code-switching, essential for the multilingual workplace settings targeted by this system. Sarvam AI's speech-to-text model is specifically designed for the phonological and morphological structure of Indian languages, as opposed to general ASR engines. The streaming adapter and adaptive chunk-size support the fast inference of API-based systems without compromises in accuracy.

6.2.2 Choice of Sarvam AI for Indian Languages

We chose the Sarvam AI platform as the multilingual speech and translation engine because it is optimised for Indian languages and code-switched speech. While general multilingual APIs consider Indian languages as add-ons, the Sarvam AI models are specifically designed for the phonological, morphological and syntactic features of Indian languages. The Mayura translation model supports formal and informal language and ensures grammatical correctness in Indian scripts without romanisation. The Bulbul TTS model synthesises natural prosody in Indian languages.

6.2.3 Choice of React and Supabase for the Web Application

React and TypeScript were chosen for the frontend due to its rich tooling for creating accessible, component-driven interfaces, and the static type checking that TypeScript provides to catch errors at compile-time, rather than runtime - essential for a system where state transitions in audio and language processing may be complicated and error-prone. Supabase was chosen as the backend system because of its PostgreSQL database, real-time subscriptions and the ability to run edge functions that run close to the user for fast AI proxying. The Radix UI component library was selected for its WCAG 2.1 accessible component library, which is essential for a product for DHH users.

6.3 Comparison with Commercial Solutions

Table 5.3 — Feature Comparison: CaptionAI vs. Commercial Solutions

Feature	CaptionAI	Google eet	Teams	Otter.ai	Rev ive
Platform- nostic	✓	✗	✗	✓	✓
On- evice privacy	✓	✗	✗	✗	✗
Speaker tribution	✓	✗	Limited	✓	✗
Sub- 0ms latency	✓	✗	✗	✗	✗
Indian nguages	✓	Partial	Partial	✗	✗
Free / no bscription	✓	✓	✓	✗	✗
Noise bustness	High	Medium	Medium	Medium	Medium

The feature comparison in Table 5.3 illustrates that the proposed CaptionAI system occupies a unique position in the solution landscape, being the only system that simultaneously provides all seven features listed. The combination of on-device privacy, platform independence, speaker attribution, sub-300-millisecond latency, and Indian language support addresses a gap that no current commercial solution fills.

6.4 Limitations

The current design has the following limitations that should be acknowledged.

First, the speech system is developed and tested primarily on English speech. The multilingual pipeline extension - although present in the web application using Sarvam AI APIs - is not part of on-device ASR pipeline. To fully support code-switched Hindi-English, Tamil-English and other common Indian language pairs in the streaming ASR pipeline requires a multi-lingual acoustic model and code-switching-aware language model, which is a major engineering enhancement.

Second, the Sarvam AI STT API is cloud-based and requires an internet connection, thus adding network latency which can exceed the 300-millisecond target on mobile networks with high latency.

Third, the speaker diarisation module system can only handle up to six speakers. Large group calls or panel discussions would need hierarchical diarisation approaches that are not yet incorporated.

Fourth, all results are estimates based on simulations and need to be tested with a fully developed prototype system in real-life settings with DHH users. No user studies have been carried out to assess comprehension accuracy, fatigue and user satisfaction.

Fifth, the system relies on access to the phone's call audio output, which is easy to access on desktop platforms but may be difficult on mobile platforms due to the sandboxing of mobile apps that prevents access to system audio from other apps.

6.5 Ethical Considerations

A system that transcribes professional phone calls involves privacy and consent issues. In the proposed system, ASR inference occurs entirely on the end user's device, with no audio or transcript content being sent to server-based resources, ensuring privacy of conversation data. Call transcripts are stored in encrypted local storage and only exported at the user's request.

But hearing users of a call may not realise that their speech is being locally transcribed by the DHH user's device. The project advises deployers to provide clear disclosure guidelines and when appropriate, a message at the beginning of the session that real-time captioning will be provided. This respects the accessibility needs of the DHH user, while meeting the transparency expectations of all call participants, and also conforms to new AI transparency standards.

The project also recognises that there may be variable accuracy across different speaker demographics. ASR systems trained primarily on "standard" varieties of English may have lower accuracy for speakers with non-standard accents, older speakers, or speakers with atypical vocal characteristics. The fine-tuning corpus has been designed to include speakers from a variety of backgrounds, however continuous monitoring and further fine-tuning will be needed to ensure that all users have equal access to the benefits of the system

CHAPTER 7: CONCLUSION AND FUTURE SCOPE

7.1 Conclusion

This capstone project has outlined the design, development, and forecasted performance of a real-time multilingual simplified captioning system for enhancing communication for the deaf and hard-of-hearing community in the workplace. This project has introduced several novel features to the accessible computing and speech technology domains.

The proposed five-stage ASR architecture, which integrates the Sarvam AI speech-to-text engine with Connectionist Temporal Classification (CTC) prefix beam search, an adaptive chunk-size mechanism for SNR-dependent optimisation, RNNoise noise suppression, GPT-2 language model rescoring, and ECAPA-TDNN-based speaker diarisation, is the first reported design to simultaneously address all four key requirements of professional captioning: captioning accuracy, captioning speed, speaker identification, and noise robustness. Theoretical predictions based on published component-level evaluation estimates show a Word Error Rate (WER) of 2.4% with clean audio and 6.1% at 5 dB SNR, with an average captioning latency of 210 milliseconds.

The companion CaptionAI Hub web application showcases the system's multilingual support in a React and TypeScript single-page application that integrates the Sarvam AI system for speech-to-text, neural machine translation and text-to-speech across eleven Indian languages. The conversational AI assistant in the application responds in the user's language, offers accessibility-sensitive advice and illustrates real-time captioning through an interactive simulation.

The dynamic chunk-size mechanism is recognised as a new design feature that reduces both captioning accuracy and latency under different levels of noise, and is able to automatically adjust the accuracy-latency trade-off in real-time. The platform-independent deployment, comprehensive on-device privacy for audio processing and Indian language code-switching altogether address requirements unmet by any existing commercial tool.

7.2 Future Scope

The project highlights five important next steps, in terms of overcoming the limitations of the current design.

7.2.1 Full Prototype Implementation and User Validation

The first step is the full deployment of the ASR pipeline as a browser extension and its testing with actual DHH users in the wild in professional environments across various domains. The user studies should assess comprehension accuracy, cognitive load, and user satisfaction with instruments used in DHH accessibility studies. These studies should inform the iterative development of the caption display, font, speaker colouring scheme and the overlay position.

7.2.2 Multilingual and Code-Switching Extension

The hardest extension to implement is the incorporation of a multilingual and code-switching-aware ASR system into the local streaming pipeline. This involves a multilingual acoustic

model for cross-linguistic recognition with mid-sentence language switching, without pre-selection on a speaker level. This is provided by the Sarvam AI speech-to-text API in the web application, but will need further development for integration into the local streaming pipeline, including training on code-switched data and extending the language model rescoring to multilingual hypothesis sets.

7.2.3 Knowledge Distillation for Mobile Devices

For a latency less than 300ms on a mid-range Android device, a lightweight on-device ASR model is needed which matches the accuracy of the Sarvam AI STT API and fits the mobile inference budget. Methods such as knowledge distillation, layer dropping, and attention head pruning should be explored, aiming for less than 5% increase in the WER compared to the full model deployed in the cloud, while adhering to the mobile inference budget.

7.2.4 Multi-Modal Enhancement via Lip Reading

Lip-reading visual information can also be used as an additional input for the ASR engine to potentially enhance noise robustness, especially at very low SNR where the acoustic signal is not enough to transcribe speech. In a video call, if a camera is available, a visual speech recognition system could be used to extract visual features of the lip movements and integrate it with the acoustic features for a multi-modal ASR system. This is an ongoing research topic and can be a future extension of the system.

7.2.5 Bidirectional Communication with Sign Language

The next step in the progression of DHH accessibility in the workplace is two-fold: the DHH user must be able to not only receive captions but also respond. Sign language recognition would allow DHH users to respond in sign language, which would be translated to text for hearing users, thus closing the loop. Sign language recognition is a different and difficult computer vision problem, but coupled with the captioning system, it would enable a full communication system that is accessible to the DHH community.

CHAPTER 8: REFERENCES

[1] World Health Organization, "World Report on Hearing," WHO Press, Geneva, Switzerland, 2021.

[2] N. Schley and C. Lindstrom, "Employment outcomes of postsecondary students with disabilities," Journal of Vocational Rehabilitation, vol. 39, no. 1, pp. 25–36, 2013.

[3] C. Punch, R. Hyde, and D. Creed, "The power of work: experiences of deaf and hard-of-hearing adults in work settings," Journal of Deaf Studies and Deaf Education, vol. 9, no. 4, pp. 435–451, 2004.

[4] R. Kushalnagar, W. Lasecki, and J. Bigham, "Captions versus transcripts for online video content," in Proceedings of the ACM W4A, Rio de Janeiro, 2013, Art. 32.

[5] J. Wald, "Creating accessible educational multimedia through editing ASR captioning in real time," Interactive Technology and Smart Education, vol. 3, no. 2, pp. 131–141, 2006.

[6] Google Cloud, "Speech-to-Text API documentation v2," Google LLC, 2024. [Online]. Available: https://cloud.google.com/speech-to-text

[7] L. R. Rabiner, "A tutorial on hidden Markov models and selected applications in speech recognition," Proceedings of the IEEE, vol. 77, no. 2, pp. 257–286, Feb. 1989.

[8] G. E. Dahl, D. Yu, L. Deng, and A. Acero, "Context-dependent pre-trained deep neural networks for large-vocabulary speech recognition," IEEE Transactions on Audio, Speech, and Language Processing, vol. 20, no. 1, pp. 30–42, Jan. 2012.

[9] A. Graves, A. Mohamed, and G. Hinton, "Deep recurrent neural networks for acoustic modelling in speech recognition," in Proceedings of ICASSP, Vancouver, 2013, pp. 6645–6649.

[10] A. Graves, S. Fernandez, F. Gomez, and J. Schmidhuber, "Connectionist temporal classification: Labelling unsegmented sequence data with recurrent neural networks," in Proceedings of ICML, Pittsburgh, 2006, pp. 369–376.

[11] D. Bahdanau, K. Cho, and Y. Bengio, "Neural machine translation by jointly learning to align and translate," in Proceedings of ICLR, San Diego, 2015.

[12] A. Gulati et al., "Conformer: Convolution-augmented transformer for speech recognition," in Proceedings of Interspeech, Shanghai, 2020, pp. 5036–5040.

[13] A. Vaswani et al., "Attention is all you need," in Proceedings of NeurIPS, Long Beach, 2017, pp. 5998–6008.

[14] A. Radford, J. W. Kim, T. Xu, G. Brockman, C. McLeavey, and I. Sutskever, "Robust speech recognition via large-scale weak supervision," in Proceedings of ICML, Honolulu, 2023, pp. 28492–28518.

[15] A. Baevski, Y. Zhou, A. Mohamed, and M. Auli, "Wav2Vec 2.0: A framework for self-supervised learning of speech representations," in Proceedings of NeurIPS, 2020, pp. 12449–12460.

[16] E. Tsunoo, Y. Kashiwagi, and S. Watanabe, "Transformer ASR with contextual block processing," in Proceedings of ASRU, Singapore, 2019, pp. 427–433.

[17] Y. Zhang et al., "Streaming end-to-end speech recognition for mobile devices," in Proceedings of ICASSP, Barcelona, 2020, pp. 6381–6385.

[18] J. Wald, "Captioning for deaf and hard-of-hearing people by editing automatic speech recognition in real time," Ph.D. dissertation, University of Southampton, UK, 2006.

[19] R. Kushalnagar et al., "Captions versus transcripts for online video content," Universal Access in the Information Society, vol. 13, no. 2, pp. 225–233, 2014.

[20] A. Cavender, R. Ladner, and E. Riskin, "MobileASL: Making sign language accessible on mobile phones," in Proceedings of ACM CHI, Florence, 2008, pp. 3075–3080.

[21] B. Desplanques, J. Thienpondt, and K. Demuynck, "ECAPA-TDNN: Emphasized channel attention, propagation and aggregation in TDNN based speaker verification," in Proceedings of Interspeech, Shanghai, 2020, pp. 3830–3834.

[22] H. Bredin et al., "Pyannote.audio: Neural building blocks for speaker diarization," in Proceedings of ICASSP, Barcelona, 2020, pp. 7124–7128.

[23] J. Barker, S. Watanabe, E. Vincent, and J. Trmal, "The fifth CHiME speech separation and recognition challenge," in Proceedings of Interspeech, Hyderabad, 2018, pp. 1561–1565.

[24] J. M. Valin, "A hybrid DSP/deep learning approach to real-time full-band speech enhancement," in Proceedings of MMSP, Vancouver, 2018.

[25] T. Mikolov, M. Karafit, L. Burget, J. Cernocky, and S. Khudanpur, "Recurrent neural network based language model," in Proceedings of Interspeech, Makuhari, 2010, pp. 1045–1048.

[26] A. Radford, J. Wu, R. Child, D. Luan, D. Amodei, and I. Sutskever, "Language models are unsupervised multitask learners," OpenAI Blog, vol. 1, no. 8, p. 9, 2019.

[27] E. J. Hu et al., "LoRA: Low-rank adaptation of large language models," in Proceedings of ICLR, 2022.

[28] S. Kim, T. Hori, and S. Watanabe, "Joint CTC-attention based end-to-end speech recognition using multi-task learning," in Proceedings of ICASSP, New Orleans, 2017, pp. 4835–4839.

[29] V. Panayotov, G. Chen, D. Povey, and S. Khudanpur, "LibriSpeech: An ASR corpus based on public domain audio books," in Proceedings of ICASSP, Brisbane, 2015, pp. 5206–5210.

[30] Sarvam AI, "Sarvam API Documentation: Speech-to-Text, Translation, and TTS," 2025. [Online]. Available: https://api.sarvam.ai

[31] Supabase, "Supabase Documentation: Edge Functions and PostgreSQL," 2025. [Online]. Available: https://supabase.com/docs

APPENDIX A: SYSTEM REQUIREMENTS SPECIFICATION

A.1 Hardware Requirements

Table A.1 — Minimum and Recommended Hardware Specifications

Component	Minimum	Recommended
Processor	Intel Core i5 (8th Gen) / pple M1	Intel Core i7 (12th Gen) / Apple 2
RAM	8 GB DDR4	16 GB DDR4 or higher
GPU	NVIDIA GTX 1060 6GB UDA 11)	NVIDIA RTX 3080 / Apple 2 Neural Engine
Storage	10 GB SSD free space	50 GB NVMe SSD
Display	1080p with 60 Hz refresh	1440p or 4K with high refresh te
Audio terface	16-bit / 16 kHz stereo audio tput	24-bit / 48 kHz with system dio capture
Network	10 Mbps stable broadband	50 Mbps or higher

A.2 Software Requirements

Table A.2 — Software Dependencies and Runtime Requirements

Software	Version	Purpose
Node.js / Bun	20.x / 1.x	JavaScript runtime for build tooling
Python	3.10+	Model inference and preprocessing scripts
CUDA Toolkit	11.8+	GPU-accelerated Sarvam AI STT inference
PyTorch	2.x	Deep learning framework for ASR and arisation
HuggingFace ansformers	4.x	Sarvam AI STT and ECAPA-TDNN model ading
WebRTC	Native owser API	Audio capture and VAD

Software	Version	Purpose
Google hrome / Firefox	Latest	Browser extension host
Supabase CLI	1.x	Edge function deployment

A.3 Supported Languages

Table A.3 — Supported Indian Languages and Their Sarvam AI Configuration

Language	BCP-Code	Unicode ock	STT upport	TTS peaker
English ndia)	en-IN	Latin	Yes	shubh
Hindi	hi-IN	U+0900–+097F	Yes	shubh
Bengali	bn-IN	U+0980–+09FF	Yes	shubh
Tamil	ta-IN	U+0B80–+0BFF	Yes	shubh
Telugu	te-IN	U+0C00–+0C7F	Yes	shubh
Kannada	kn-IN	U+0C80–+0CFF	Yes	shubh
Malayalam	ml-IN	U+0D00–+0D7F	Yes	shubh
Marathi	mr-IN	U+0900–+097F	Yes	shubh
Gujarati	gu-IN	U+0A80–+0AFF	Yes	shubh
Punjabi	pa-IN	U+0A00–+0A7F	Yes	shubh
Odia	od-IN	U+0B00–+0B7F	Yes	shubh

APPENDIX B: GLOSSARY OF TERMS

ASR: Automatic Speech Recognition — the computational process of converting spoken audio into written text.

BHM: Bayesian Hidden Markov Model — a probabilistic model used for online speaker clustering in diarisation.

CTC: Connectionist Temporal Classification — a sequence-to-sequence loss function enabling end-to-end ASR training.

DER: Diarisation Error Rate — the fraction of speech-active time incorrectly attributed to speakers.

DHH: Deaf and Hard-of-Hearing — the clinical and social designation for individuals with significant hearing loss.

ECAPA-TDNN: Emphasised Channel Attention, Propagation and Aggregation Time-Delay Neural Network — a speaker embedding model.

GMM: Gaussian Mixture Model — a probabilistic model used in classical HMM-based ASR acoustic modelling.

HMM: Hidden Markov Model — a statistical model of temporal sequences used in classical speech recognition.

LUFS: Loudness Units Full Scale — a standardised loudness measurement unit per ITU-R BS.1770.

LoRA: Low-Rank Adaptation — a parameter-efficient fine-tuning technique for large neural networks.

LM: Language Model — a probabilistic model of word sequences used to rescore ASR hypotheses.

MFCC: Mel Frequency Cepstral Coefficient — a compact acoustic feature representation for speech signals.

SNR: Signal-to-Noise Ratio — the ratio of signal power to background noise power, measured in decibels.

TTS: Text-to-Speech — the computational synthesis of spoken audio from written text.

VAD: Voice Activity Detection — the process of identifying speech-active and silence periods in an audio stream.

WER: Word Error Rate — the fraction of words incorrectly transcribed, the standard ASR accuracy metric.

WCAG: Web Content Accessibility Guidelines — the international standard for web accessibility published by W3C.

VoIP: Voice over Internet Protocol — technology for transmitting voice communications over internet connections.